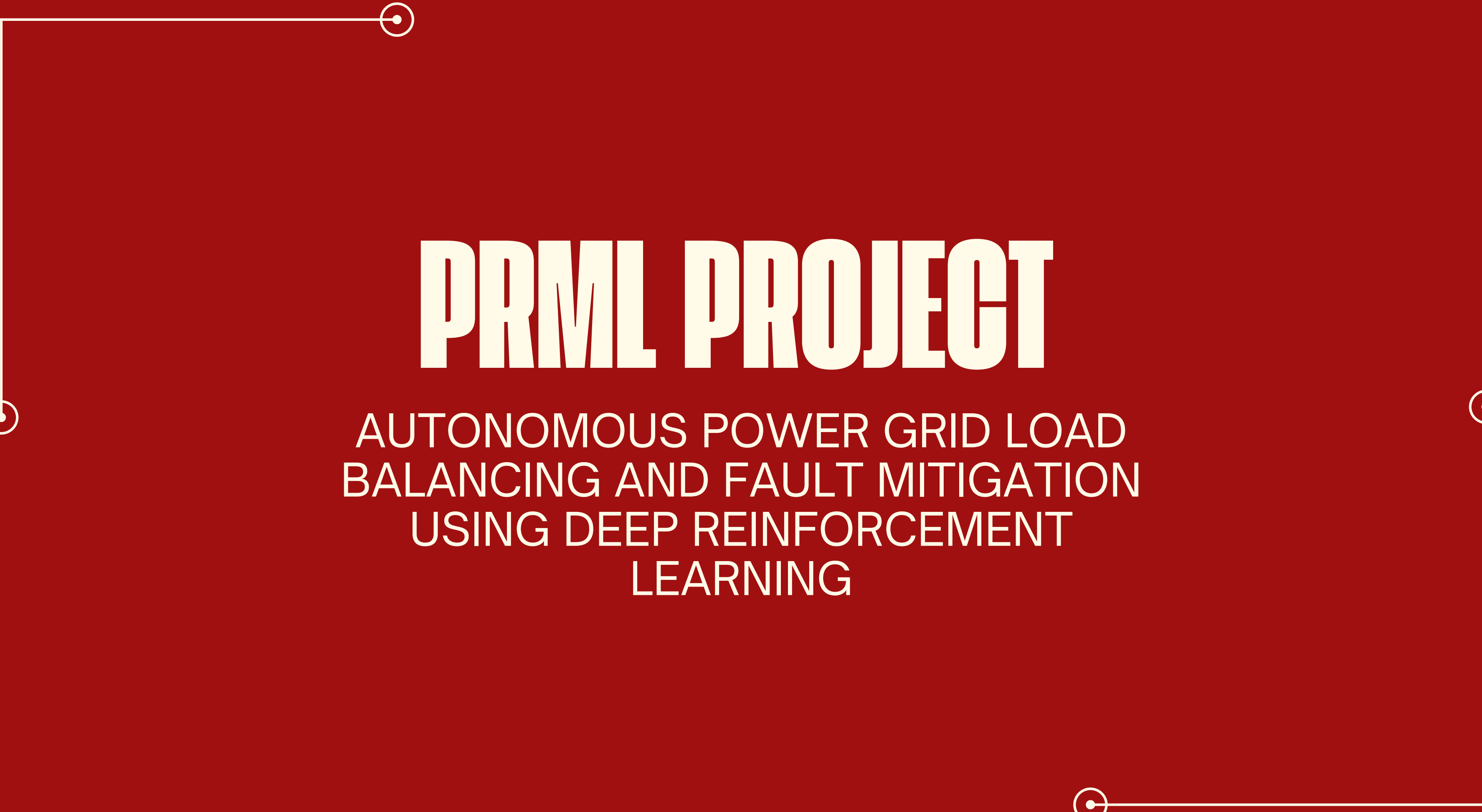


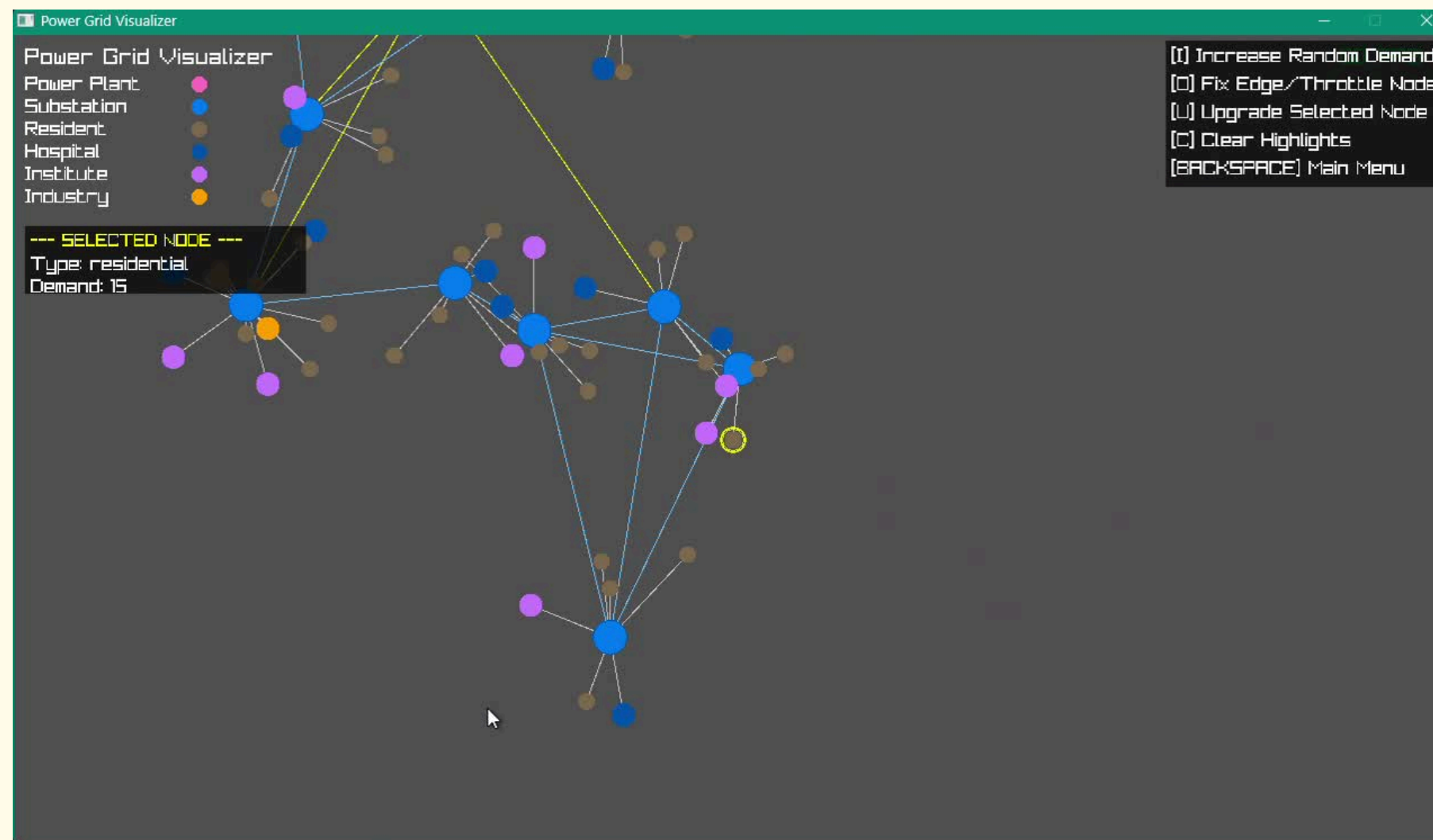


PRML PROJECT

AUTONOMOUS POWER GRID LOAD
BALANCING AND FAULT MITIGATION
USING DEEP REINFORCEMENT
LEARNING

Problem Statement

Power distribution grids face dynamic, time-varying demand across multiple substations, leading to overloads, inefficiencies, and cascading failures.



- Physics-based simulator using DC power flow (fluid-like model); operates in a continuous state/action space for smooth load balancing and control.
- Models grid as pressure-flow system (θ , F , x relations), enabling realistic power distribution and overload mitigation.

- Graph-based simulator using directed weighted networks; decisions are made over nodes and edges of the grid.
- Uses discrete actions with graph algorithms (BFS, path-finding, adjacency traversal) for topology-aware control.

Problem Statement

Sequential Decision-Making via Deep RL

Objective Learn an optimal control policy $\pi(a|s)$ that minimizes overload events and power loss while maintaining grid stability across a ≥ 50 substation network.

Models: DDDQN & SAC Dueling Double DQN for discrete switching decisions; Soft Actor-Critic for continuous control — both trained via DRL.

System Environment A stochastic, time-dependent ≥ 50 -node power grid modeled as an MDP with high-dimensional state space and temporal dependencies.

MDP Formulation

State (s)
Grid voltage, load demand, fault flags, substation status

Action (a)
Load redistribution, circuit switching, control setpoints

Reward (r)
Stability score - overload penalty - efficiency loss

State



Agent



Action



Reward

Dataset Description

8,760

Samples

Hourly readings

50

Features

Substations

1 Year

Duration

Jan-Dec 2024

Numerical

Data Type

Continuous (MW)

Dataset Info

Source:

Synthetic Substation Power Grid (1-yr simulation, 2024)

Sampling Freq:

Hourly (8,760 timestamps × 50 substations)

Target Variable:

Active power demand per substation (MW), hourly

Data Characteristics

No missing values

Continuous, noise-free time-series

Strong temporal patterns (daily & weekly & seasonal)

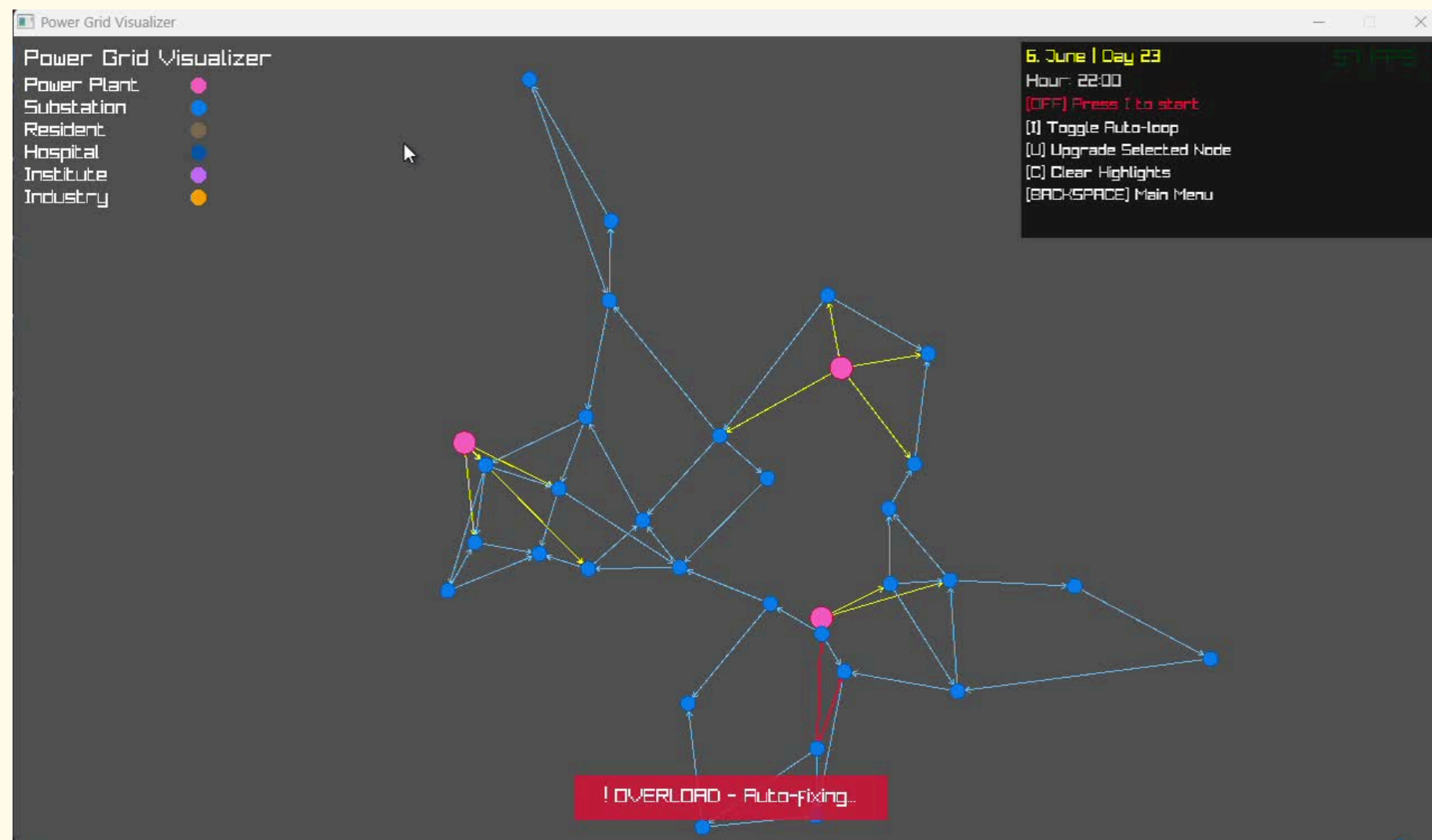
Key Features

Feature	Zone Type	Avg Demand (MW)
Sub_01_Urb	Urban	~32.0
Sub_02_Ind	Industrial	~20.8
Sub_05_Rur	Rural	~4.7
Sub_18_Com	Commercial	~26.0
Sub_38_Ind	Industrial	~32.5
Sub_49_Rur	Rural	~3.3

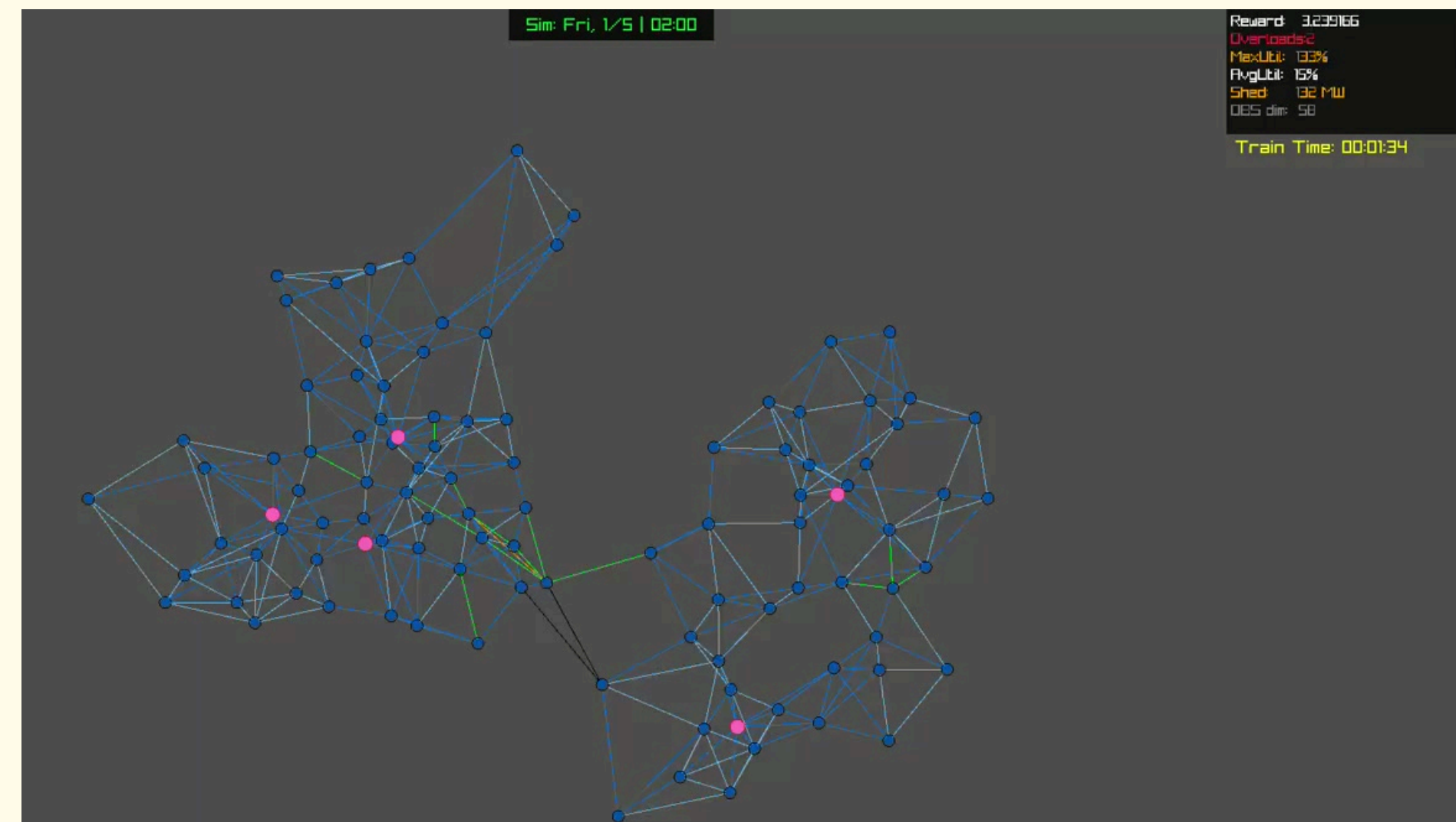
- For sac the dataset is taken from this csv file
- For dqn , the data is generated by a custom function which itself takes care of the temporal trends

TWO MODELS

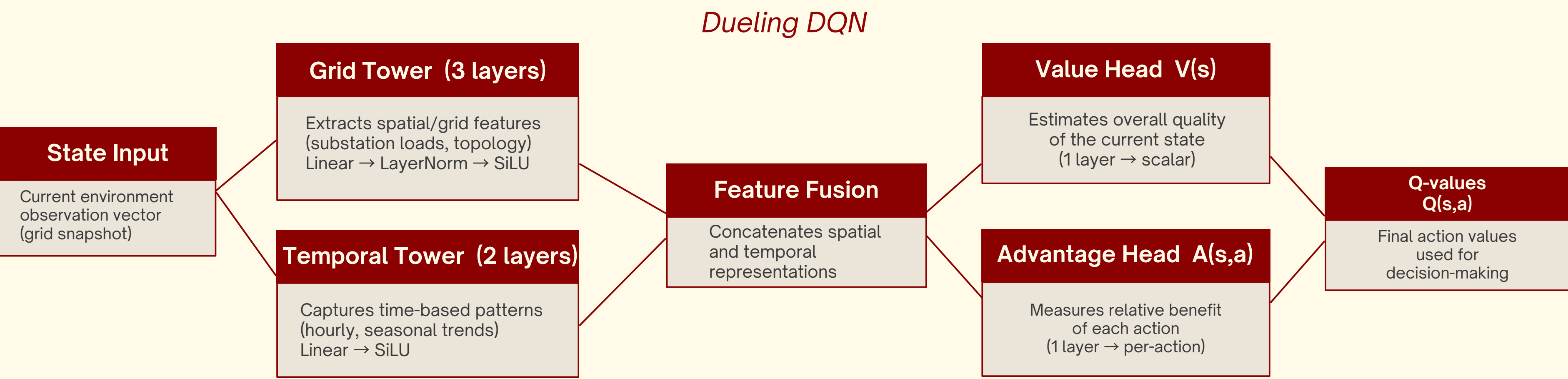
DQN (Deep Q-Network)



SAC (Soft Actor Critic)



Dueling Double Deep Q-Network (DDDDQN) Architecture



Double DQN is a Q-learning algorithm that uses two neural networks — an Online network that selects the best action ($a^ = \operatorname{argmax}_a Q_{\text{online}}(s', a)$) and a Target network that evaluates its value ($Q_{\text{target}}(s', a^*)$). The target Q-value is computed as $\text{target} = r + \gamma \cdot Q_{\text{target}}(s', \operatorname{argmax}_a Q_{\text{online}}(s', a))$.*

Prioritized Replay

Samples high-error transitions more often. Faster learning from mistakes.

Huber Loss

Quadratic for small errors; linear for large spikes. Robust to reward outliers.

Activation Function

SiLU
 $f(x) = x \cdot \sigma(x)$
Smooth, no dead neurons, better than ReLU in RL.

Double Network

Online proposes action; Target evaluates it. Reduces overestimation bias.

Action Name	What It Does	Reward (Overloaded)	Reward (Healthy)
THROTTLE_RESIDENTIAL_5%	Cuts residential demand by 5%	−0.25	+10
THROTTLE_RESIDENTIAL_10%	Cuts residential demand by 10%	−0.50	+10
THROTTLE_RESIDENTIAL_20%	Cuts residential demand by 20%	−1.00	+10
THROTTLE_INDUSTRIAL_5%	Cuts industrial demand by 5%	−0.15	+10
THROTTLE_INDUSTRIAL_10%	Cuts industrial demand by 10%	−0.30	+10
THROTTLE_INDUSTRIAL_20%	Cuts industrial demand by 20%	−0.60	+10
REROUTE_5%	Shifts 5% of worst overloaded edge load	−10 / +15	+15
REROUTE_10%	Shifts 10% of worst overloaded edge load	−10 / +15	+15
REROUTE_20%	Shifts 20% of worst overloaded edge load	−10 / +15	+15
UPGRADE_NODE_10%	Increases worst substation max_limit by 10%	+20	−10
UPGRADE_NODE_20%	Increases worst substation max_limit by 20%	+20	−10
UPGRADE_NODE_30%	Increases worst substation max_limit by 30%	+20	−10
LOAD_SHED_5%	Cuts all residential+industrial by 5%	−25	+10
LOAD_SHED_10%	Cuts all residential+industrial by 10%	−25	+10
LOAD_SHED_20%	Cuts all residential+industrial by 20%	−25	+10
IDLE	Does nothing — grid state untouched	−50	+5

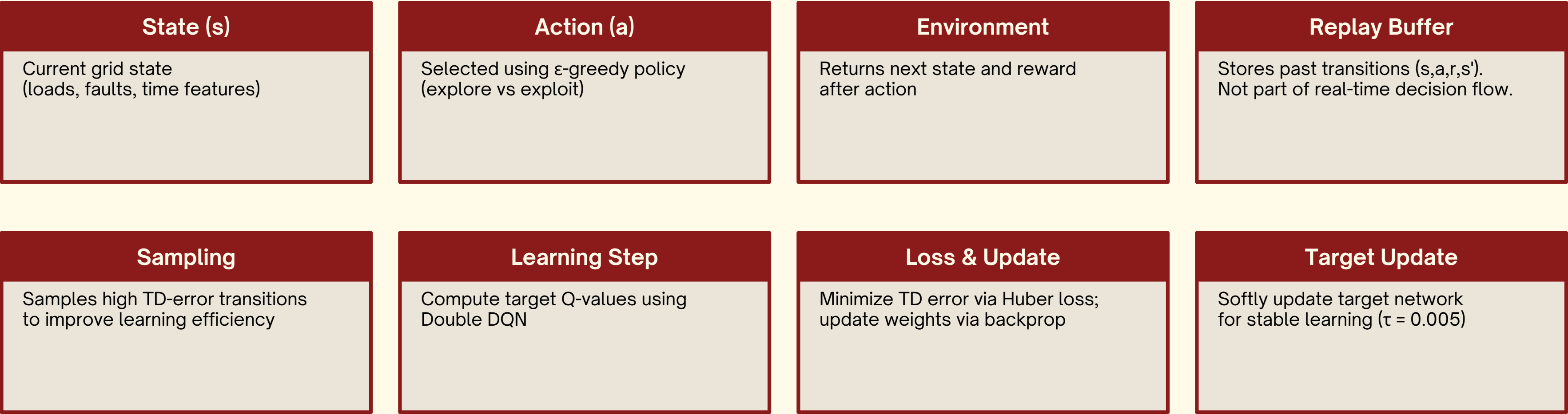
REWARD FORMULA

base_penalty = (−50 × n_overloaded_edges) + (−20 × n_unmet_nodes)

total_reward = base_penalty + action_specific_reward

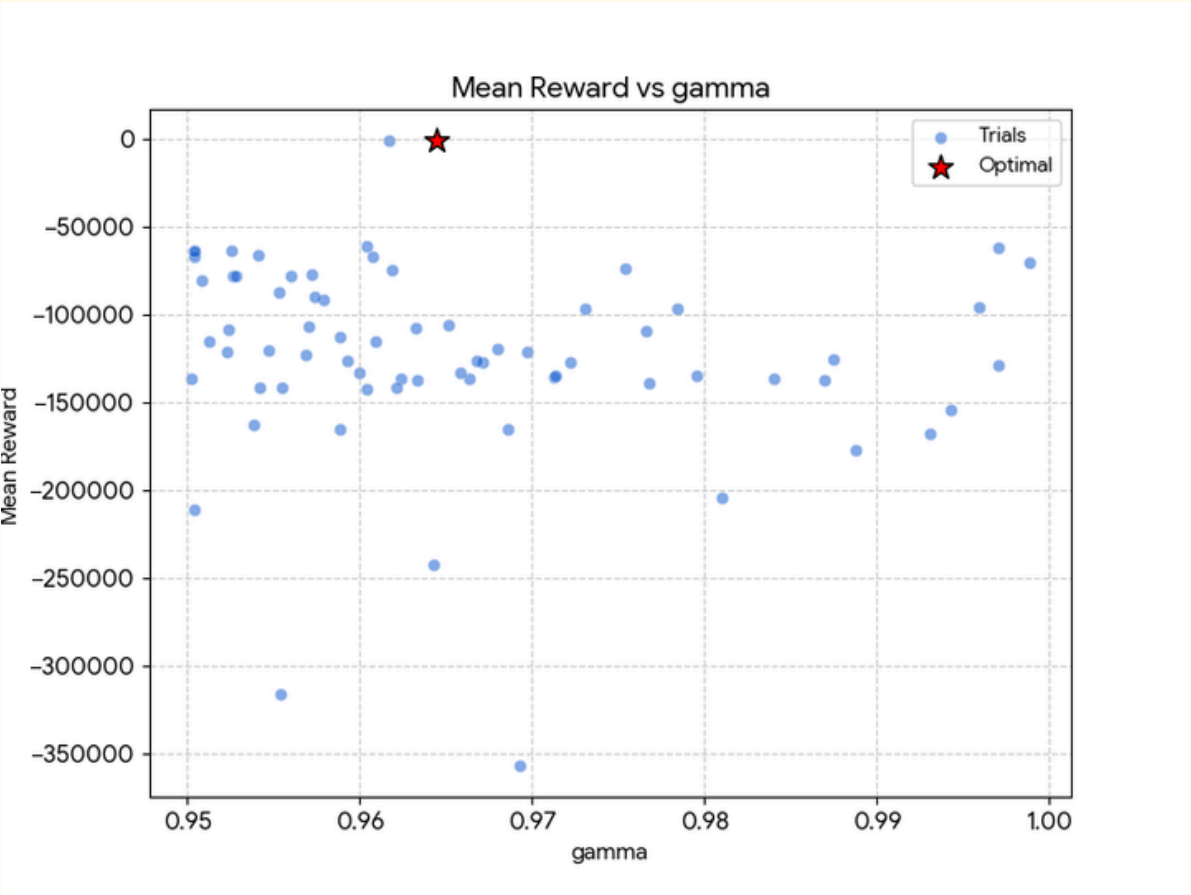
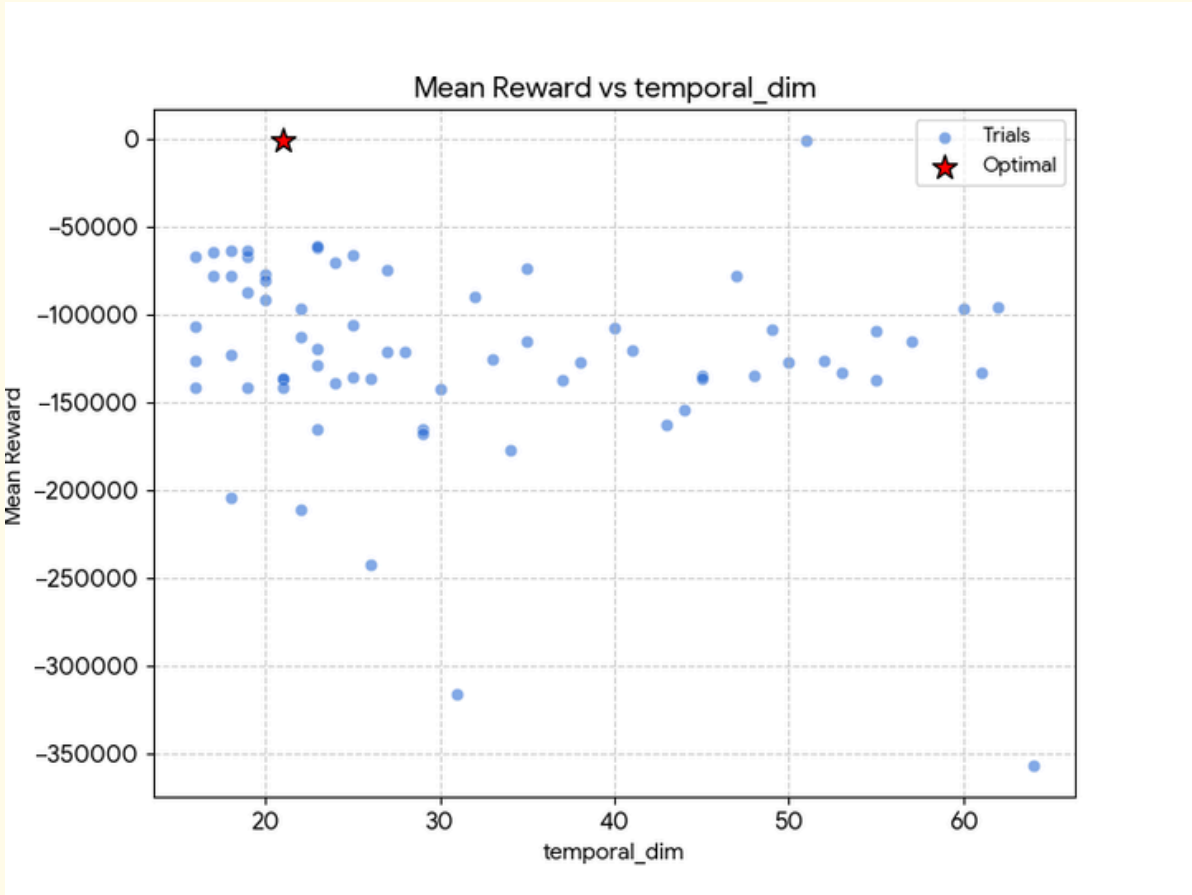
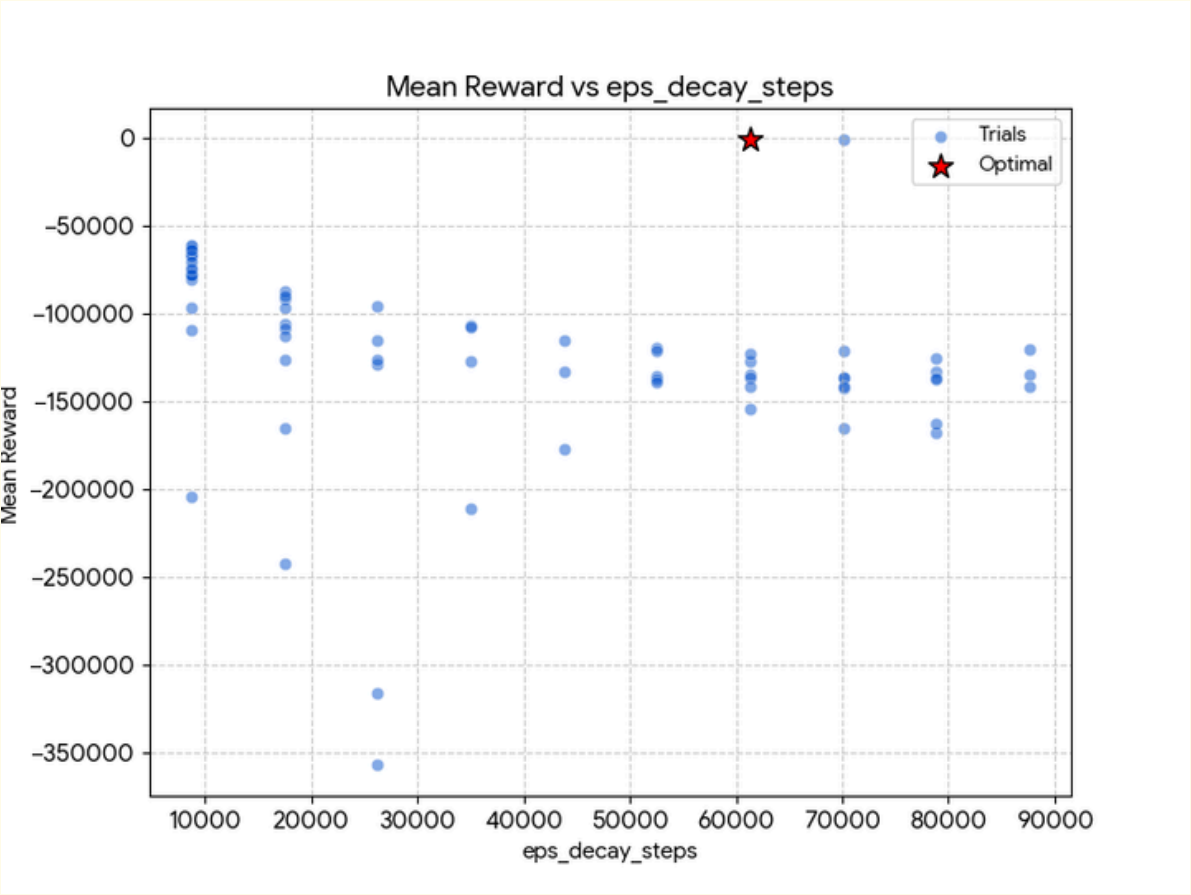
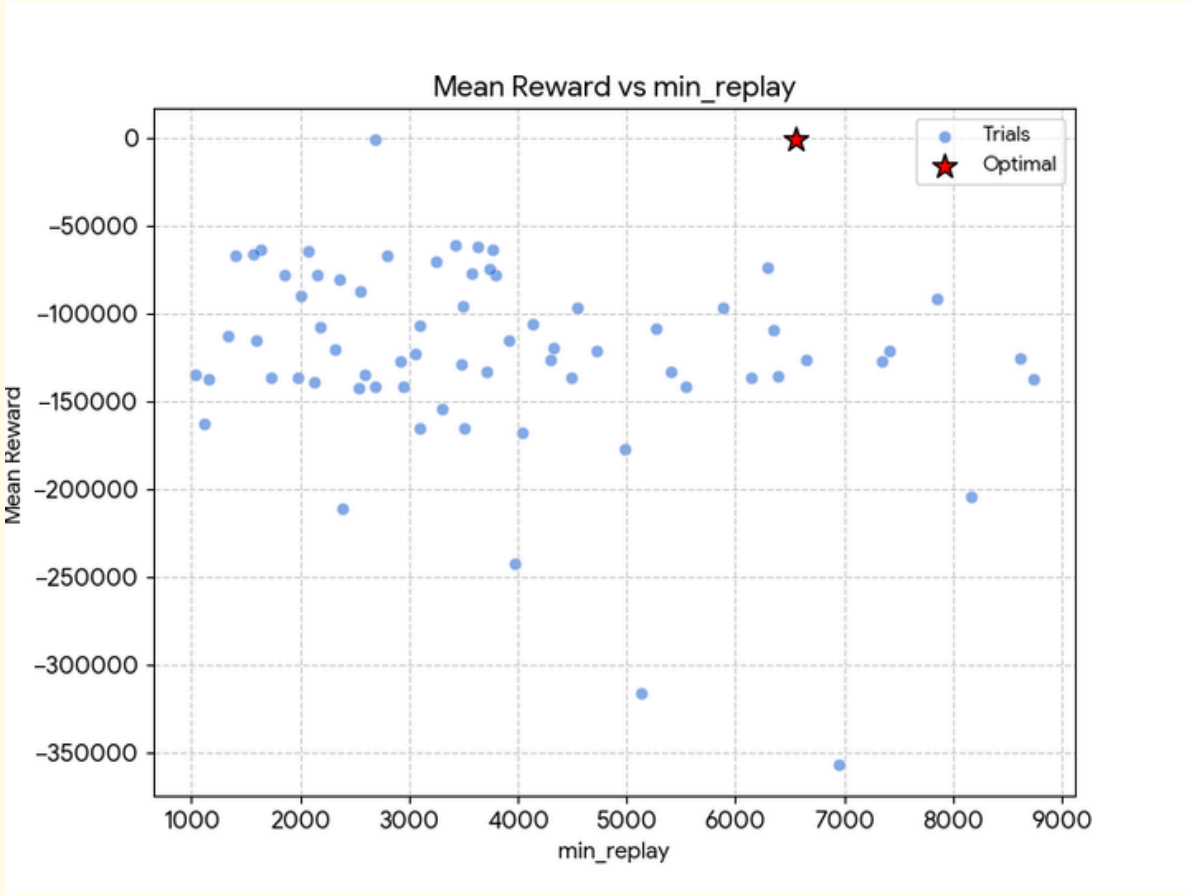
DQN Training Process

The agent learns an optimal policy by interacting with the environment and updating Q-values based on observed rewards.



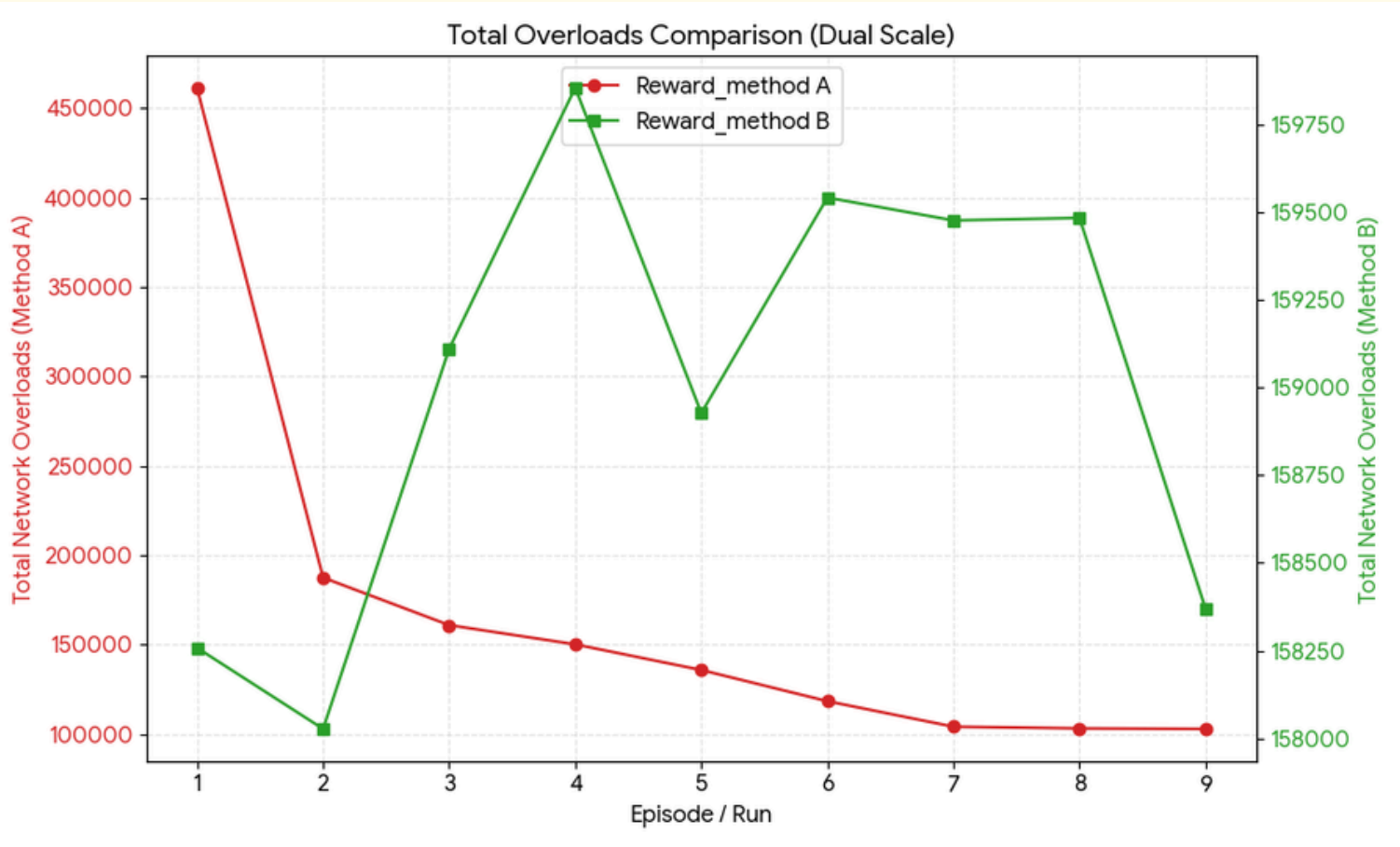
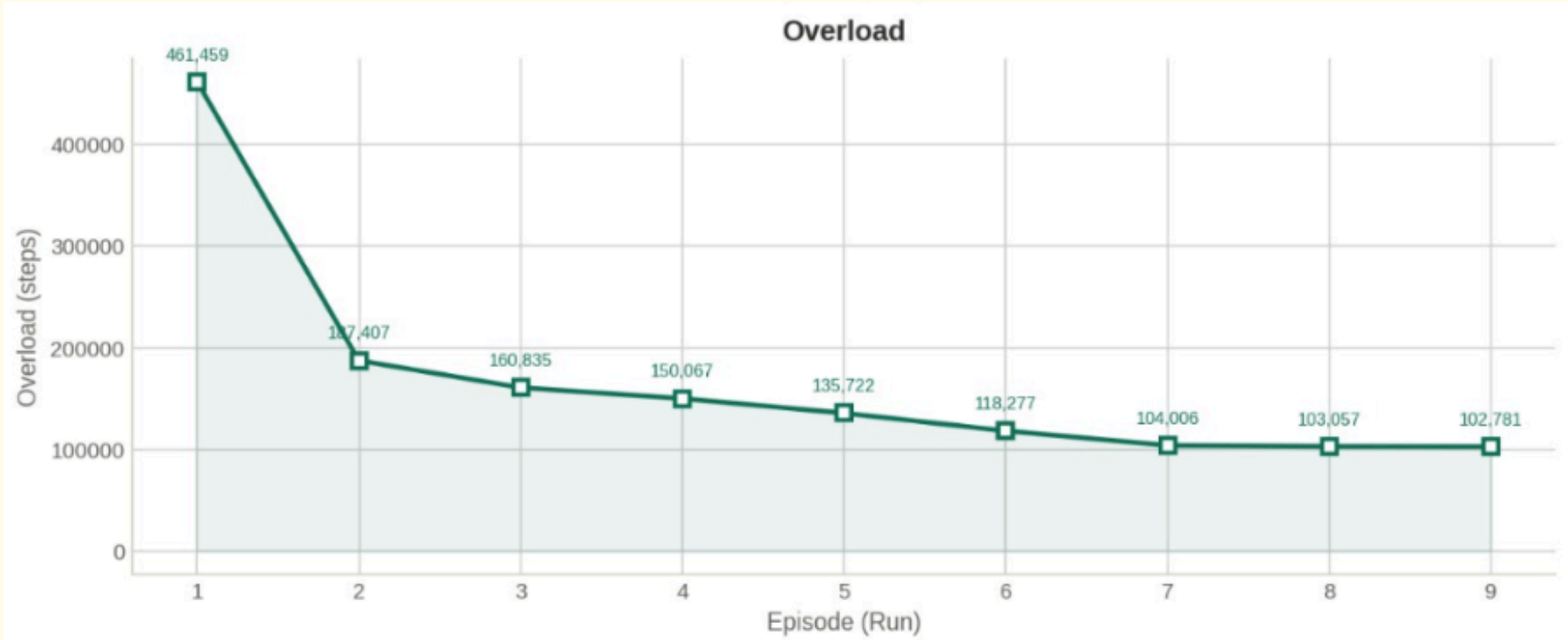
Training repeats over multiple episodes until convergence.

Hyperparameter tuning and selection of optimal parameters for training



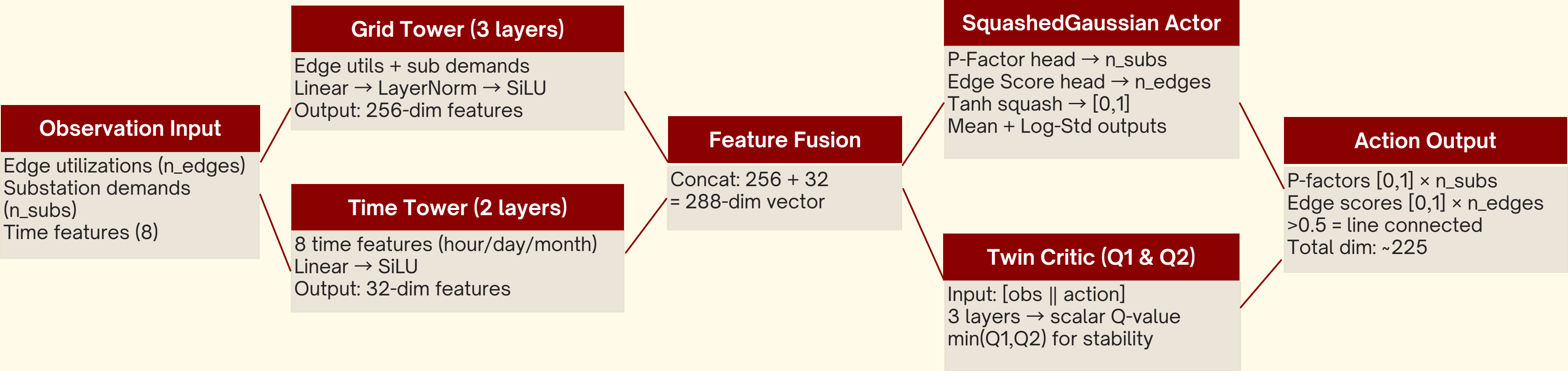
Category	Parameter	Value
Core Learning	lr	3e-4
	gamma	0.955
	batch_size	256
Training Stability	target_update_tau	0.005
	train_freq	5
	replay_capacity	100000
Exploration	eps_start / eps_end	1/0.03
	eps_decay_steps	8760
Architecture	hidden_dim	256
	n_layers	5
	temporal_dim	23
PER	per_alpha	0.55
	per_beta_start/end	0.35/1

RESULT AND ANALYSIS OF DQN MODEL



Soft Actor-Critic (SAC) Architecture

Three neural networks work together to learn optimal continuous power grid control policies.



Entropy Temp α (Auto-tuned)	Key Design Choices	Weight Initialization	Total Parameters
Target entropy: $-\text{dim} \times 0.005$ $\alpha_{\text{min}} = 0.10$, $\alpha_{\text{max}} = 2.0$ Balances exploration vs exploit	SiLU activations throughout LayerNorm for input stability Reparameterization trick	P-factor bias: 1.5 $\rightarrow \sim 98\%$ served Edge bias: 0.85 $\rightarrow \sim 98\%$ connected Log-std bias: -2.0	Grid Tower: $\sim 200\text{K}$ Twin Critics: $\sim 300\text{K}$ each Full SAC agent: $\sim 900\text{K}$

ACTION SPACE

Action Vector = [p_factors (100) | edge_scores (125)]

P-Factors [0,1]

Controls demand served per substation. 1.0 = full demand; 0.5 = 50% load shedding

Edge Scores [0,1]

>0.5 = line connected; ≤0.5 = line disconnected (current_load = 0)

REWARD COMPONENTS (7 terms, clamped to [-10, +10])

r_serve	+4.0 × served_ratio	[0, +4]
r_overload	-10 × trip_ratio - 15 × sev ²	(-∞, 0]
r_danger	-2.0 × avg_danger_sev ²	(-∞, 0]
r_cascade	-6.0 × trip_ratio ²	[-6, 0]
r_health	+0.3 × healthy_ratio	[0, +0.3]
r_shed	-4.0 × shed_ratio × (1-trip_ratio)	[-1.5, 0]
r_topology	-0.5 × disconnected_ratio	[-0.5, 0]

ACTION PROCESSING PIPELINE

- 1 Python clips action to [0, 1]; splits into p_factors + edge_scores
- 2 C++ applies p_factors → substation.per; runs demand scaling
- 3 C++ checks edge_score > 0.5 → connect(); else disconnect()
- 4 DC power flow solver computes current_load on all active lines
- 5 Reward function evaluates 7 components from resulting grid state

REWARD DESIGN PHILOSOPHY

- | | |
|--------------------------|--|
| 1 Safety First | Overload penalties dominate — thermal limits must not be breached |
| 2 Service Second | r_serve is primary positive driver; maximize demand satisfaction |
| 3 Context-Sensitive Shed | Shedding only penalised when grid is safe (scales with 1-trip_ratio) |
| 4 Cascade Superlinear | r_cascade uses trip_ratio ² — multiple failures punished harder |
| 5 Smooth Gradients | Quadratic terms + linear terms give SAC stable learning signal |

SAC TRAINING PROCESS

The agent learns by continuously interacting with the power grid simulator and updating three networks.

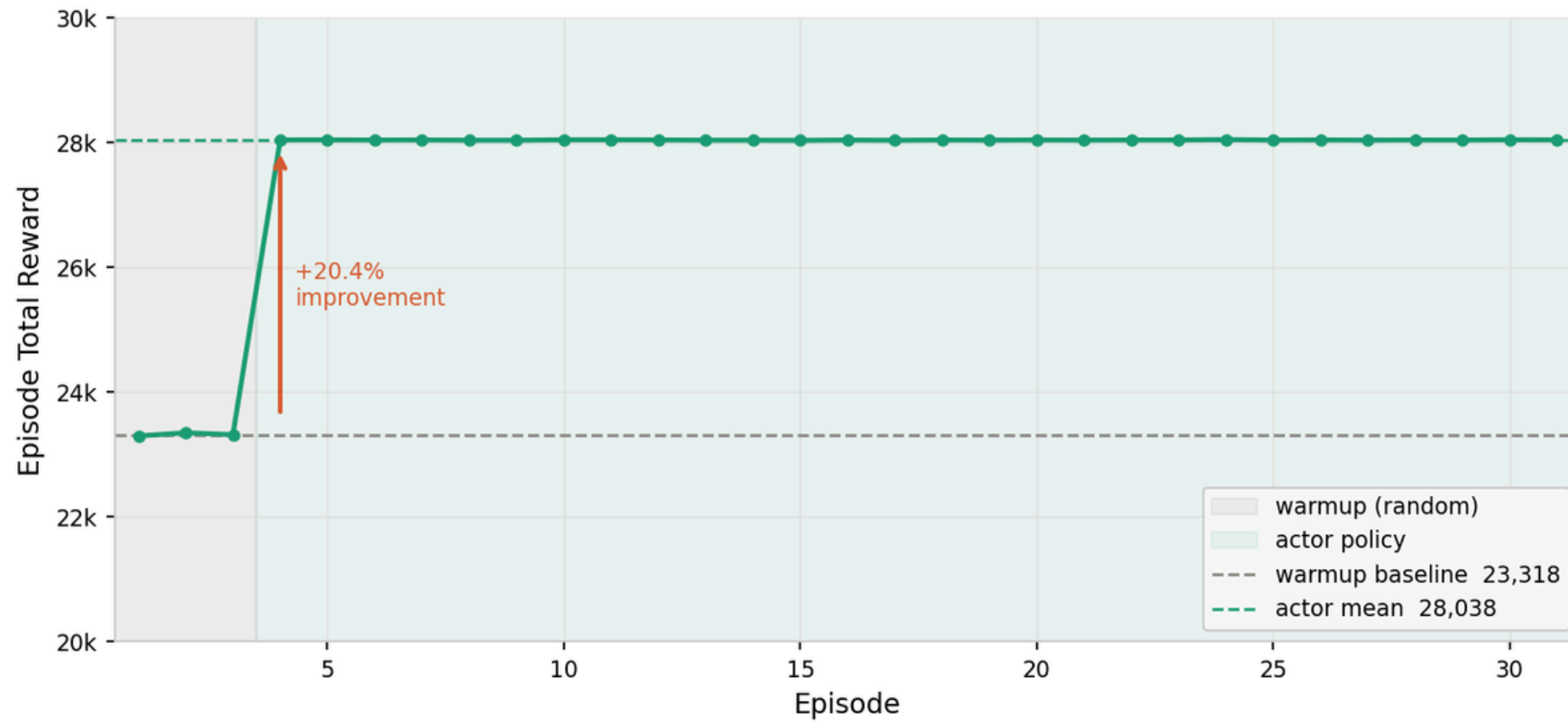
Observation (s)	Action Selection	Environment Step	Replay Buffer
Grid state: edge utils, substation demands, time features	Warmup: Beta(9,1) random Training: stochastic policy Eval: deterministic mean	C++ power grid simulator Returns: next obs, reward, terminated/truncated	Capacity: 200,000 transitions Min before training: 26,280 Batch size: 256
Critic Update	Actor Update	Alpha Update	Soft Target Update
Sample batch from buffer Compute Bellman target: $y = r + \gamma \cdot (Q_target - \alpha \cdot \log \pi)$	Maximize soft Q-value: $J = Q(s,a) - \alpha \cdot \log \pi(a s)$ Reparameterization gradient	Auto-tune entropy temp Target: $-\dim \times \text{entropy_scale}$ Clamped to [0.10, 2.0]	EMA update of target critics $\tau = 0.005$ each step Stabilizes Q-value targets

Warmup: 26,280 random steps → Active Learning: SAC updates every step → Best model saved per episode reward.

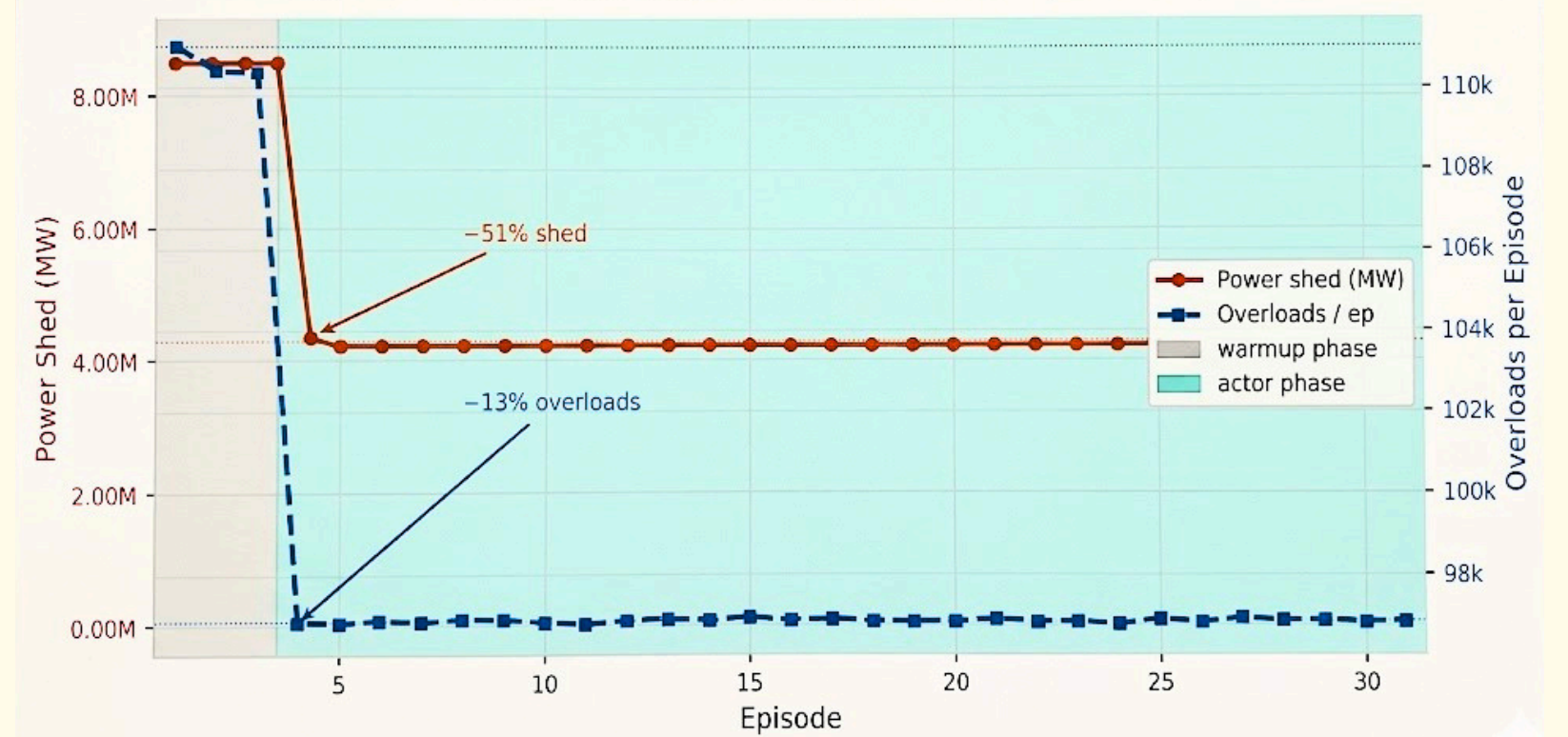
Category	Core Learning				Stability			Exploration		Replay Buffer		Architecture
Parameter	critic_lr / actor_lr	alpha_lr	gamma	batch_size	tau	grad_clip	reward_scale	entropy_scale	alpha_min / alpha_max	replay_capacity	min_replay / warmup	hidden_dim
Value	3e-4 / dynamic	1e-4	0.99	256	0.005	1.0	0.05	0.005	0.10 / 2.0	200,000	26,280	256

RESULT AND ANALYSIS OF SAC MODEL

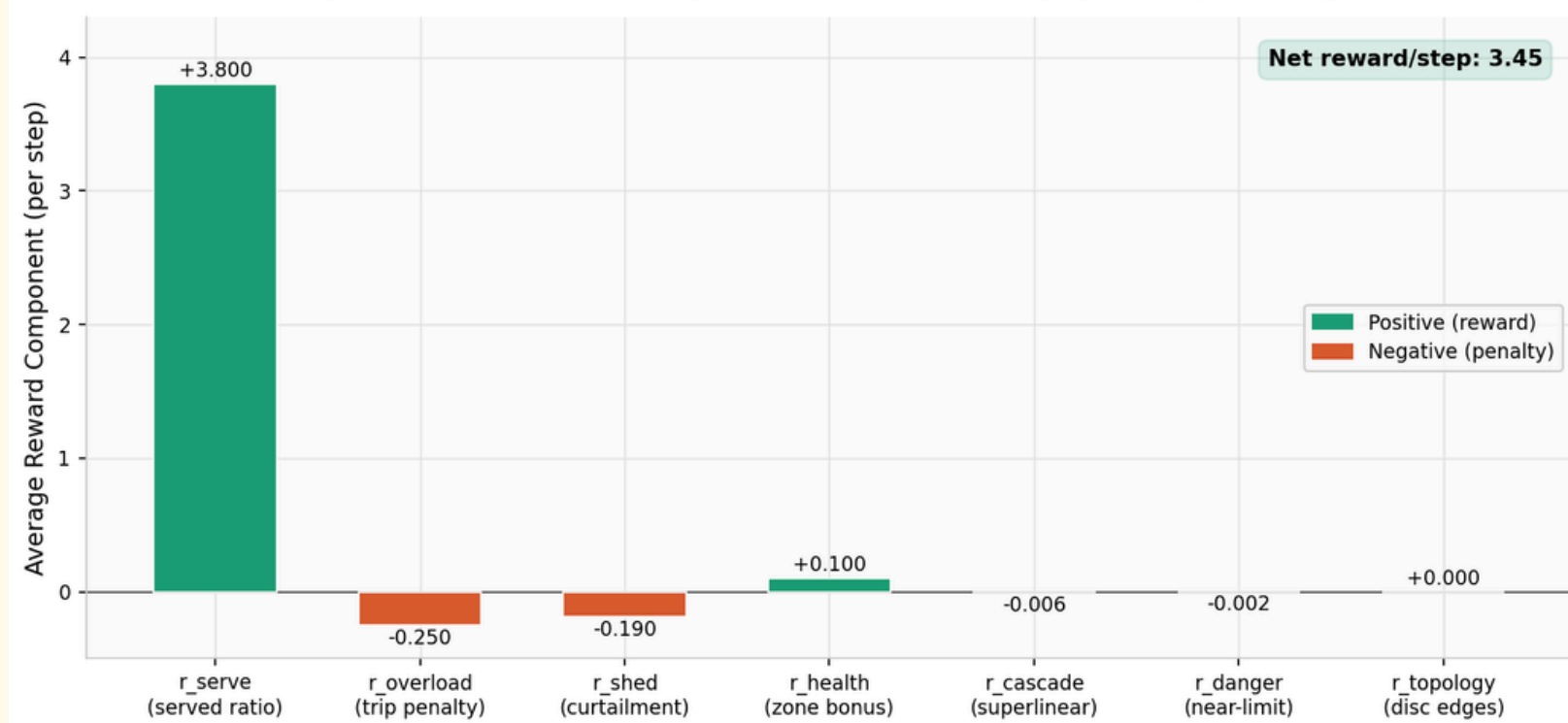
Graph 1 — Episode Reward Curve (Run 4)



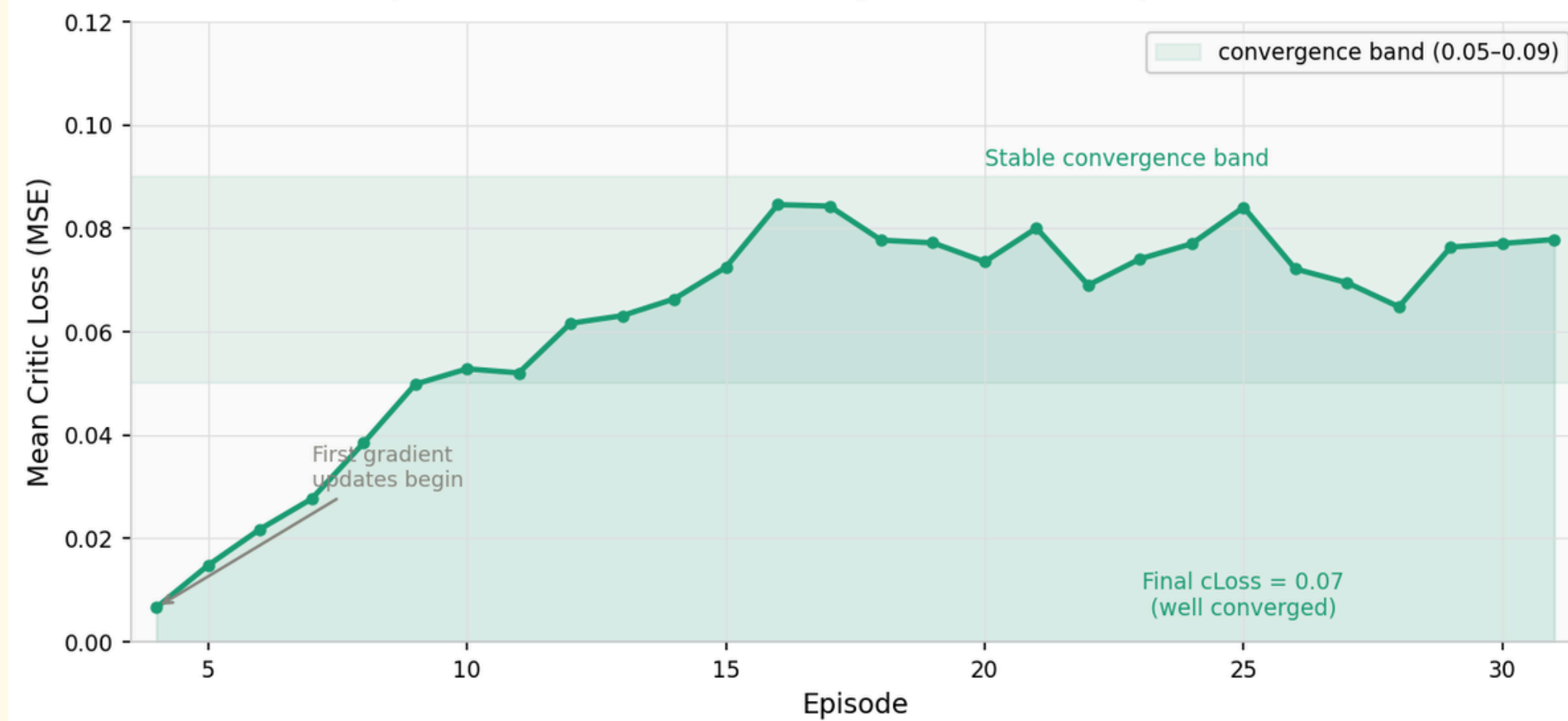
Graph 2 — Power Shed and Overloads Over Episodes (Run 4)



Graph 4 — Reward Decomposition — Final Policy (per-step averages)

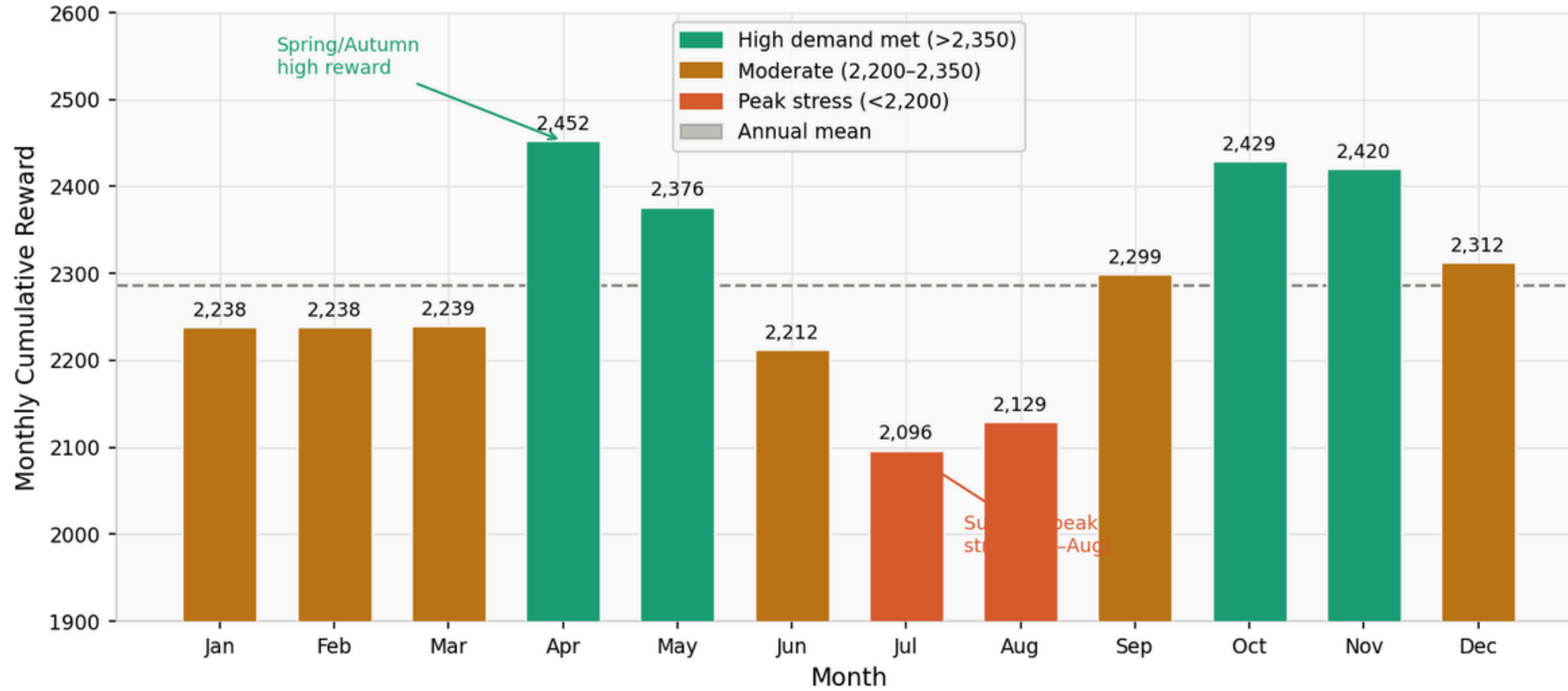


Graph 3 — Critic Loss Convergence (Run 4, Episodes 4-31)



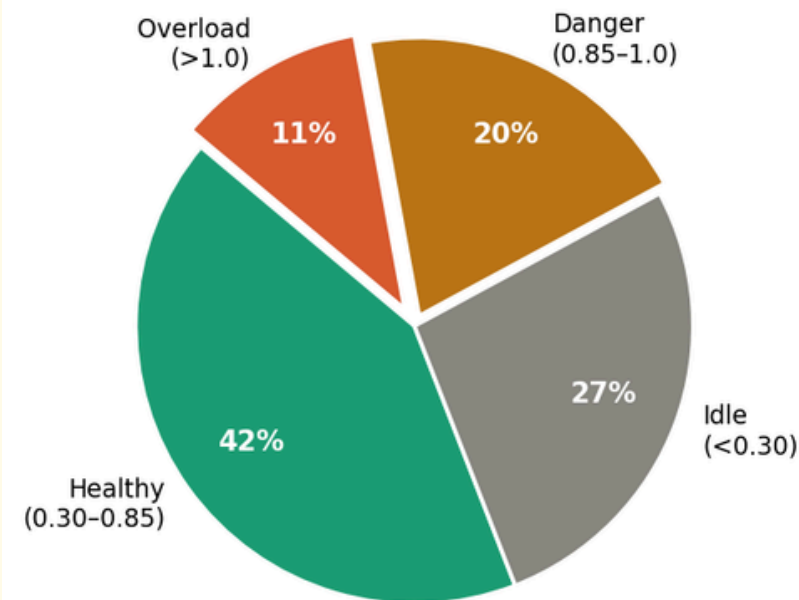
RESULT AND ANALYSIS OF SAC MODEL

Graph 5 — Seasonal Reward Pattern (Run 4, Episode 4)

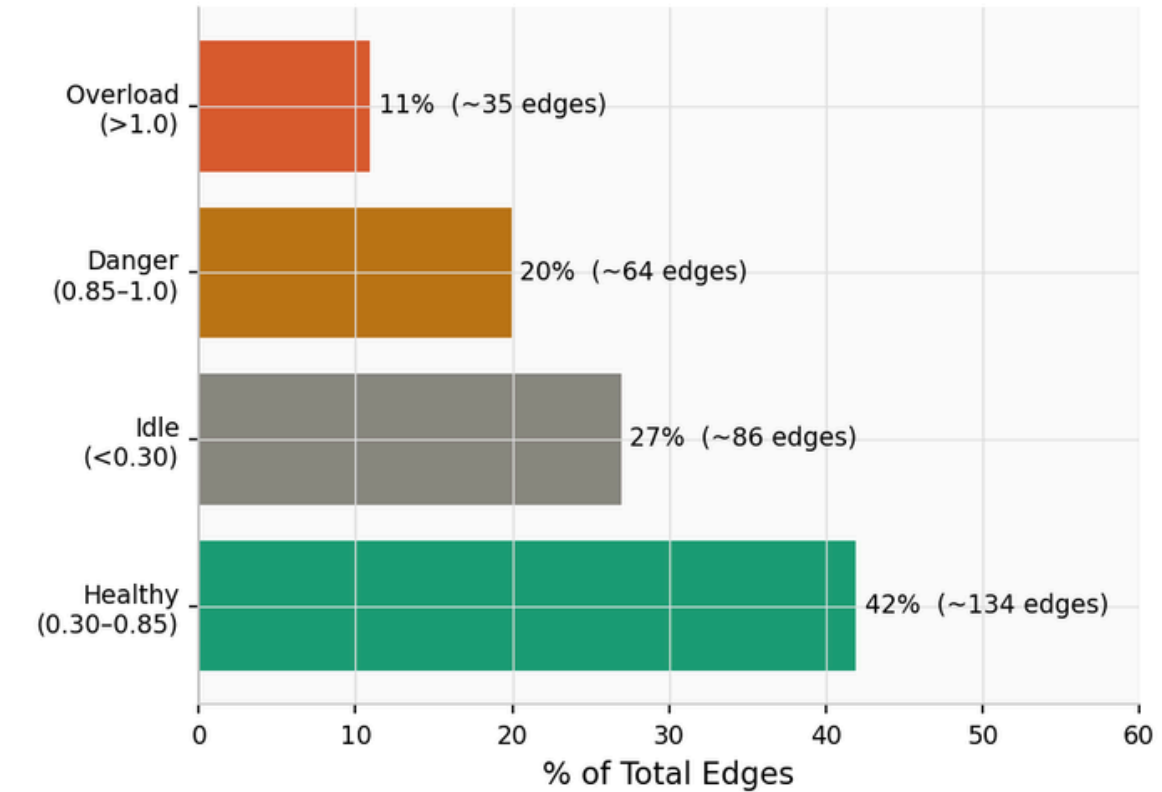


Graph 6 — Edge Utilisation Zones — Final Policy

Edge Zone Distribution
(% of all 319 edges, avg per step)



Same Data — Absolute Context
(319 edges total)



WORK DIVISION

Simulator Modification for Reinforcement Learning - Adapted and enhanced the simulation environment to support RL by defining state space, action space, and reward mechanisms, along with implementing environment dynamics and constraints.
By Parv Dixit (B24EE1093) and Harry Singh (B24EE1101)

DQN Model Development - Designed and implemented the Deep Q-Network (DQN) model, including network architecture, experience replay, and training pipeline for decision-making in discrete state-action environments.
By Harry Singh (B24EE1101) and Krish Jain (B24EE1097)

SAC Model Development - Developed the Soft Actor-Critic (SAC) model with actor-critic architecture, incorporating entropy-based optimization for efficient learning in continuous action spaces.
By Parv Dixit (B24EE1093) and Parayush Kanphade (B24EE1052)

THANK YOU!